

CodeRisk Agent

AI-Powered Code Security Analysis
Running Entirely on Local AMD GPU

Project Specification Document

AMD AI DevMaster Hackathon — Track 2: Agentic AI

Team: Yang Weike (Captain) + Iolo (AI Assistant)

Repository: github.com/a9320/code-risk-agent

Version: 1.0 | July 2026

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Application Scenarios
- 4. System Architecture
- 5. The 4 Agents
- 6. Core Capabilities
- 7. Model & Local Deployment
- 8. AMD ROCm GPU Optimization
- 9. Performance Benchmarks
- 10. Compliance & Security
- 11. Testing
- 12. Future Work

1. Executive Summary

CodeRisk Agent is an AI-powered code security analysis system that performs deep vulnerability detection entirely on local AMD Radeon GPUs. Unlike traditional static analysis tools (Semgrep, Clang-Tidy) that rely on pattern matching, CodeRisk Agent uses LLM-driven semantic understanding to reason about code logic, trace attack paths, and provide exploitability evidence.

The system employs a multi-agent architecture with 4 specialized agents coordinated by an Orchestrator state machine. A self-reflection loop enables the system to discover vulnerabilities that other tools miss, while a dual-memory system (correct + error patterns) allows it to improve with each scan.

All LLM inference runs locally on AMD Radeon GPUs via ROCm/HIP. No source code ever leaves the machine. This makes CodeRisk Agent suitable for enterprises with strict compliance requirements (HIPAA, GDPR, SOX).

2. Problem Statement

Enterprises need AI-powered code security analysis, but cannot upload source code to cloud services. Compliance regulations (HIPAA, GDPR), intellectual property concerns, and corporate policies all prohibit sending proprietary code to external APIs like OpenAI Copilot or GitHub Copilot.

Existing solutions fall into two categories:

Category	Example	Limitation
Traditional Static Analysis	Semgrep, Clang-Tidy	Pattern matching only — misses logic bugs, no reasoning
Cloud AI	GitHub Copilot, Snyk	Source code leaves the machine — compliance risk

CodeRisk Agent solves this by combining the depth of AI analysis with the security of local execution. It understands code logic (like cloud AI) but runs entirely on local AMD GPU (like static analysis tools).

3. Application Scenarios

Enterprise Code Audit

Security teams scan proprietary codebases before releases. CodeRisk Agent identifies vulnerabilities that pattern-matching tools miss, with full evidence chains for compliance reporting. [Agent 1 + Agent 2 + Agent 3]

CI/CD Pipeline Integration

Automated security scanning in continuous integration. The system outputs SARIF format, compatible with GitHub Actions, GitLab CI, and Jenkins. [Agent 1 + Agent 2, CI/CD compatible]

Developer Self-Service

Individual developers scan code before committing. The system provides concrete fix suggestions that developers can copy directly into their code. [Agent 1 + Agent 2 + Agent 4]

Compliance & Regulatory

Industries with strict data residency requirements (finance, healthcare, government) can run AI-powered code analysis without sending code to external services. [Full pipeline, zero external calls]

Security Training

The evidence chain and attack scenario generation make CodeRisk Agent useful for teaching developers about secure coding practices. [Agent 3 evidence chain + Agent 4 reports]

4. System Architecture

CodeRisk Agent follows a pipeline architecture managed by an Orchestrator state machine. The pipeline has 5 phases, each handled by specialized agents.

4.1 Pipeline Overview

INIT → PARSE → ANALYZE → VERIFY → REPORT → DONE

Phase	Component	Description
Phase 0: PARSE	File Collector	Traverses directory, detects language by extension (.c, .h, .py)
Phase 1: ANALYZE	Agent 1: Static Analyzer	27 detection rules (C:13, Python:14), CPU parallel (4 threads)
Phase 1.5	Dependency Scanner	Local OSV data + fallback dictionary, 9 common packages
Phase 2: ANALYZE	Semgrep + Taint Analyzer	Semgrep CLI integration + variable data-flow tracking
Phase 3: VERIFY	Agent 2: Semantic Analyzer	LLM-driven analysis on GPU, CoT prompts, parallel (configurable workers)
Phase 4: VERIFY	Agent 3: Deep Verifier	Triple cross-validation + self-reflection loop, local CVE DB

4.2 Data Flow

Each agent produces Risk objects with structured evidence. Risks flow through the pipeline: Agent 1 discovers patterns → Semgrep + Taint Analysis enriches → Agent 2 validates with LLM → Agent 3 cross-checks with tools + knowledge base → Agent 4 formats reports. The Orchestrator manages state transitions and error handling.

5. The 4 Agents

5.1 Agent 1: Static Analyzer

Responsibility: Pattern-based vulnerability detection using 27 rules.

Category	Rules	CWE Coverage
C Dangerous Functions	8	CWE-120, CWE-134, CWE-78
C Vulnerability Patterns	5 (2 original + 3 new)	CWE-476, CWE-415, CWE-22, CWE-327, CWE-190
Python Dangerous Calls	14 (10 + 2 patterns + 2 new)	CWE-95, CWE-502, CWE-611, CWE-377, CWE-328, CWE-798

5.2 Agent 2: Semantic Analyzer

Responsibility: LLM-driven code understanding. Runs on AMD GPU via llama-server HTTP API.

- Chain-of-Thought (CoT) prompting for structured reasoning
- Validates Agent 1 findings — confirms true positives, identifies false positives
- Discovers new vulnerabilities missed by pattern matching
- Generates attack scenarios ("if input X, then result Y")
- JSON structured output with automatic retry (up to 3 attempts)

5.3 Agent 3: Deep Verifier

Responsibility: Triple cross-validation with self-reflection loop.

Three validation sources:

- Tool confirmation — Semgrep, taint analysis results
- CWE knowledge base — 15 MITRE ATT&CK techniques mapped to CWEs
- Local CVE database — 69,686 CVE entries from NVD (SQLite)

Self-reflection: After initial verification, Agent 3 asks "Did we miss anything?" and re-analyzes each file with additional context. This discovers vulnerabilities that the initial pass missed.

5.4 Agent 4: Report Generator

Responsibility: Multi-format output with actionable insights.

Supported formats:

- JSON — machine-readable, CI/CD integration
- Markdown — human-readable reports with CWE/CVE links
- SARIF 2.1.0 — industry standard for static analysis tools
- Rich Terminal — colored output with interactive details

6. Core Capabilities

6.1 Triple Cross-Validation

Each risk is validated against three independent sources: tool output, CWE knowledge base, and local CVE database. This reduces false positives and increases confidence in findings.

6.2 Self-Reflection Loop

Agent 3 performs multiple verification rounds. In round 1, it validates existing findings. In round 2, it asks "What did we miss?" and discovers additional risks. In testing, the self-reflection loop consistently found 4 additional risks per analysis.

6.3 Dual Memory System

The system maintains two types of memory in JSON format:

Memory Type	Purpose	Effect
Correct Memory	Stores confirmed true positives	Boosts confidence for repeated patterns
Error Memory	Stores confirmed false positives	Suppresses known false positives in future scans

Memory activates after 2+ scans. The system tracks "seen X times" for each pattern, enabling faster and more accurate detection over time.

6.4 Evidence Chain

Every risk includes: source code snippet, line numbers, CWE classification, reasoning process, and conclusion. This provides full traceability — unlike cloud AI which operates as a black box.

6.5 CVE Integration

The system queries a local SQLite database of 69,686 CVE entries (built from NVD feeds) to associate discovered vulnerabilities with known CVEs. This provides real-world context and severity ratings (CVSS scores).

7. Model & Local Deployment

7.1 Model Selection

Parameter	Value
Model	Qwen2.5-Coder-32B-Instruct
Quantization	Q4_K_M (GGUF format)
Size	~19 GB
Context Window	128K tokens

7.2 Deployment Architecture

The LLM runs via llama-server (compiled from llama.cpp with HIP backend) as a separate HTTP service. The CodeRisk Agent connects to it via HTTP API (default: localhost:8080). This separation allows independent scaling and model swapping.

7.3 Two Backend Options

Backend	Use Case	Config
local_http	Recommended — llama-server HTTP API	LOCAL_HTTP_URL=http://localhost:8080
local_llama_cpp	Alternative — direct GGUF load via Python	LOCAL_MODEL_PATH=/path/to/model.gguf

7.4 Environment Setup

One-time setup commands:

```
# Clone repository git clone https://github.com/a9320/code-risk-agent.git cd code-risk-agent # Create
virtual environment python3 -m venv .venv && source .venv/bin/activate pip install -e . # Download
model hf download Qwen/Qwen2.5-Coder-32B-Instruct-GGUF \ qwen2.5-coder-32b-instruct-
q4_k_m.gguf --local-dir /workspace/models # Build CVE database python
scripts/download_cve_data.py --years 2023 2024 2025 2026 # Compile llama.cpp with HIP cd
/workspace/llama.cpp ROCM_PATH=/opt/rocm cmake -B build -DGGML_HIP=ON -
DLLAMA_BUILD_SERVER=ON cmake --build build --config Release -j$(nproc) # Start llama-server
./build/bin/llama-server -m /workspace/models/qwen2.5-coder-32b-instruct-q4_k_m.gguf \ -ngl 999 -fa
1 --port 8080 & # Run analysis export LLM_BACKEND=local_http export
LOCAL_HTTP_URL=http://localhost:8080 python3 main.py analyze tests/test_cases/ --output terminal
```

8. AMD ROCm GPU Optimization

8.1 HIP Backend

llama.cpp is compiled with the HIP backend (GGML_HIP=ON), which enables GPU inference on AMD Radeon GPUs via ROCm. The key flag that changed in 2026 is GGML_HIP=ON (replacing the older GGML_HIPBLAS=ON).

8.2 GPU Layer Offloading

The -ngl 999 flag offloads all model layers to GPU memory. For the 32B Q4_K_M model (~19GB), this requires approximately 19.6GB of VRAM on the AMD Radeon Pro W7900 (48GB).

8.3 Flash Attention

The -fa 1 flag enables Flash Attention, which significantly speeds up attention computation on AMD GPUs. This reduces memory usage and improves throughput.

8.4 ROCm Version

Tested on ROCm 7.2.4. The HIP backend requires ROCm 7.x or later. In our testing on an older container environment (ROCm 7.2.1), performance was significantly reduced (~2.9 t/s). Upgrading to ROCm 7.2.4 resolved this, achieving 29.5 t/s.

9. Performance Benchmarks

All benchmarks were measured on AMD Radeon Pro W7900 (48GB VRAM) with ROCm 7.2.4.

9.1 LLM Inference Speed

Metric	CPU	AMD GPU (HIP)	Speedup
Token Generation	6.8 t/s	105 t/s	15.4×
Prompt Processing	~40 t/s	628 t/s	15.7×
VRAM Usage	—	19.6 GB / 48 GB (41%)	—

9.2 End-to-End Analysis

Metric	Value
Test Files	5 (C + Python vulnerability samples)
Risks Detected	47-48
Total Analysis Time	~8 minutes (32B, ROCm 7.2.4)
LLM Calls	Variable (Agent 2 + Agent 3 rounds)
Total Tokens	Variable (depends on code size)

9.3 Agent Timing Breakdown

Agent	Time	Percentage
Agent 1: Static Analysis	0.004 s	0.0%
Semgrep + Taint Analysis	7.2 s	1.2%
Agent 2: LLM Semantic	177 s (~3 min)	28.5%
Agent 3: Deep Verification	438 s (~7.3 min)	70.4%
Agent 4: Report Generation	0.04 s	0.0%

The LLM inference (Agent 2 + Agent 3) accounts for 99% of total analysis time. GPU acceleration is critical — on CPU, the same analysis would take over 2 hours.

10. Compliance & Security

10.1 Zero External Network Calls

CodeRisk Agent makes zero external network requests at runtime. All components run locally:

Component	Data Source
LLM Inference	Local GGUF model via llama-server (HIP backend)
CVE Data	Local SQLite database (pre-downloaded from NVD)
OSV Data	Local JSON index (pre-downloaded) + fallback dictionary
CWE/ATT&CK Knowledge	Local Python dictionaries
Report URLs	Reference links only — opened by user, not by system

10.2 Reference Links in Reports

URLs that appear in analysis reports (CWE references, CVE links, MITRE ATT&CK links) are clickable reference links for the user's convenience. They open in the user's browser and are never called by the system itself.

10.3 Data Preparation

Before first use, vulnerability databases must be built from public data sources. This is a one-time setup step that downloads data from NVD and OSV bulk feeds. No API keys are required. After setup, all data is stored locally.

11. Testing

The project includes 51 unit tests covering all major components:

Test File	Tests
test_static_analyzer.py	13
test_llm_client.py	15
test_schemas.py	10
test_memory.py	7
test_cve_client.py	6

Test cases include 5 vulnerability samples: `buffer_overflow.c`, `command_injection.c`, `memory_issues.c`, `code_injection.py`, and `sql_injection.py`.

12. Future Work

- Language expansion: Java, Go, Rust, JavaScript/TypeScript
- Cross-function taint analysis via call graph construction
- IDE integration: VS Code extension for real-time scanning
- Custom rule authoring: user-defined detection patterns
- Incremental analysis: only scan changed files in CI/CD
- Model fine-tuning: train on project-specific vulnerability patterns